```
sale = (user, product) ->
  auditLog "Sold #{product} to #{user}"
  # Some other stuff happens here
```

Then Agtron tells you that you also need to log refunds, so you run along and add an `auditLog` there too:

```
refund = (user, product) ->
  auditLog "Refund for #{product} to #{user}"
  # Some other stuff happens here
```

Then Agtron tells you that you also need to log ... you know where this is headed *and* that it will quickly get tedious. What if instead you could add the logging code from the outside so that you could create a new `auditedRefund` function easily?

```
auditedRefund = withAuditLog refund
```

Or perhaps even assign a new function to the existing `refund` variable?

```
refund = withAuditLog refund
```

You can do this with the help of `before`. First, you need to implement it:

```
before = (decoration) ->
  (base) ->
    (params...) ->
      decoration params...
      base params...
```

If you aren't familiar with these dots, they're for rest and spread parameters and they help you deal with an arbitrary number of arguments. You'll learn more about them in the next chapter.

Now you can define the `withAuditLog` function:

```
withAuditLog = before (params...) ->
  auditLog params...
```

You could also implement `withAuditLog` to use `after` instead of `before`. The difference is only in the order in which the functions are invoked. The definition for `after` is similar to that for `before`:

```
after = (decoration) ->
  (base) ->
    (params...) ->
```